PYTHON FUNDAMENTALS

Theory and Programming Practice for Freshers

A beginner practice pack covering core Python syntax, decisions, loops, number logic, strings, lists, dictionaries, nested data, and a complete stationery shop capstone.

Inside

30 theory questions | 40 focused programming tasks | 1 integrated capstone project

Work through the focused exercises first, then build the capstone using the same concepts.

Part 1: Theory Questions

Answer in your own words. For tracing questions, write the exact result and explain why.

Getting Started

1. What is a variable? Give two valid variable names and one invalid name.

2. What is the difference between `int`, `float`, `str`, and `bool`?

3. What type does `input()` return? Why might conversion be needed?

4. Explain the difference between `/`, `//`, `%`, and `**`.

5. What is an expression? Give one arithmetic and one comparison example.

6. What does `type(value)` tell you?

7. What is the difference between concatenating strings and adding numbers?

Strings and Decisions

8. Explain indexing and slicing. What does `text[1:4]` select?

9. Name three useful string methods and describe each.

10. What is the difference between == and =?

11. What values do `and`, `or`, and `not` produce? Give one example of each.

12. When should you use `if`/`elif`/`else`?

13. What is a nested `if` statement? Give a situation where it helps.

14. What is a conditional expression? When can it reduce readability?

Loops and Number Logic

15. What is the difference between a `for` loop and a `while` loop?

16. What causes an infinite loop? How can a loop be stopped safely?

17. What is an accumulator? Show how a running total is updated.

18. What is a counter? How is it different from an accumulator?

19. Why might nested loops be useful? Give one example.

20. What is a Boolean condition? Give a condition that checks whether a number is positive and even.

Lists and Dictionaries

21. What is a list? How do you access, update, append, and remove an element?

22. What does `len(a_list)` return? What error may occur when an index is out of range?

23. What is the difference between `append()` and assigning to a list index?

24. What is a dictionary? How are keys and values related?

25. How do `keys()`, `values()`, and `items()` differ?

26. When is `dict.get(key)` safer than `dict[key]`?

27. Can a dictionary key be repeated? What happens if you assign a value to an existing key?

28. What does it mean for a list to contain dictionaries? Give a real-world example.

29. Trace the result: `a = [2, 4]; b = a; b.append(6)`. What is a and why?

30. What is the difference between copying a list with `b = a` and `b = a.copy()`?

Part 2: Programming Tasks

Create a separate Python file for each exercise. Read each task carefully, then validate normal, boundary, and invalid inputs where requested.

Focused Tasks 01-30

01. Personal introduction

Ask for a person's name, city, and favorite subject. Print a neat three-line introduction. Use meaningful variable names and at least one `print` call with `sep` or an f-string.

Extension: Print a one-line version with a custom separator.

02. Type detective

Read a name, age, height in metres, and whether the student has prior coding experience (`yes`/`no`). Convert the numeric inputs to suitable types. Print each value and its type. Convert the yes/no answer to a Boolean value.

Check: Age is an integer; height can include decimals.

03. Temperature converter

Read a temperature in Celsius and display Fahrenheit and Kelvin. Use `F = C * 9 / 5 + 32` and `K = C + 273.15`. Format results to one or two decimal places.

Extension: Reject a temperature below absolute zero (-273.15 C).

04. Two-number calculator

Read two numbers and a menu choice for addition, subtraction, multiplication, division, or remainder. Print a readable equation. Handle an invalid menu choice and prevent division/remainder by zero.

Extension: Add floor division and exponentiation.

05. Name card

Read a full name. Print it in uppercase, lowercase, title case, and with leading/trailing spaces removed. Show the character count before and after trimming.

Extension: Print the first and last character when the cleaned name is non-empty.

06. Username builder

Read first name, last name, and birth year. Build a lowercase username from the first name and last name, with a dot between them and the final two digits of the birth year. Print the original components and final username.

Extension: Replace internal spaces in names with underscores.

07. Message inspector

Read a sentence and a search word. Report its length, first and last character (if present), whether the search word occurs, and the first five and last five characters using slicing. Handle an empty sentence.

Extension: Count how many times the search word occurs, ignoring case.

08. Number comparison

Read two integers. Report whether the first is greater, smaller, or equal. Also report whether both are positive, whether either is zero, and whether they are different. Use comparison and logical operators.

09. Grade and pass decision

Read marks from 0 to 100. Print a grade band (A: 90-100, B: 75-89, C: 60-74, D: 40-59, F: below 40) and pass/fail status. Reject values outside the valid range. Use an `if`/`elif` chain.

10. Ticket eligibility (nested conditions)

Ask for age and whether the person has a student ID. Under 5: free entry. Ages 5-17: child ticket. Ages 18+: adult ticket; apply a student discount only when a valid student ID is present. Reject negative age and display the final ticket price using your own stated prices.

Extension: Add a senior discount and ensure discounts do not stack.

11. Leap year and compact decision

Read a year and determine if it is a leap year using the Gregorian rules: divisible by 400, or divisible by 4 but not 100. First write the full Boolean condition; then write the same decision using a conditional expression and compare readability.

12. Multiplication table (`for` loop)

Read an integer and print its multiplication table from 1 through 12. Align the output as equations.

Extension: Let the user choose the ending multiplier.

13. Count and sum (`while` loop)

Keep asking for integers until the user enters 0. Print how many non-zero values were entered, their sum, and their average. Do not divide when no values were entered.

Extension: Track the largest value as well.

14. Menu loop

Create a repeating menu with options: greet, show help, and quit. Continue until quit is selected. Handle unknown choices without ending the program.

Extension: Count the number of menu actions before quitting.

15. Running balance

Start with a user-entered balance. Repeatedly accept deposit, withdrawal, or quit. Reject non-positive amounts and withdrawals larger than the current balance. Show the balance after each valid transaction.

16. Nested loops: class attendance grid

Ask for the number of classes and students per class. For each student, ask for a name and attendance (`present`/`absent`). Print one class summary at a time, including present and absent counts. Use nested loops and counters.

Extension: Display a simple row of `P` and `A` markers for each class.

17. Digit sum and reverse

Read a non-negative integer. Using a loop and arithmetic only (no string conversion), calculate the sum of its digits and its reversed value. Treat 0 correctly.

18. Factorial

Read a non-negative integer and compute its factorial with a loop. Define `0!` as 1. Reject negative input.

Extension: Also print each multiplication step.

19. Prime checker

Read an integer and report whether it is prime. Numbers below 2 are not prime. Test possible divisors with a loop and stop early once a divisor is found.

Extension: List all primes from 2 through a user-entered limit.

20. Fibonacci sequence

Read how many Fibonacci terms to display. Print that many terms beginning with 0, 1. Handle 0 and 1 term requests correctly; reject a negative count.

21. Star patterns

Read a positive row count and print (a) a left-aligned triangle and (b) an inverted left-aligned triangle using nested loops. Build each row by repetition or an inner loop.

22. Number patterns

Read a positive row count. Print a right-triangle pattern where row 1 contains `1`, row 2 contains `1 2`, and so on. Then print a second pattern where each row repeats its row number.

23. Shopping list (list basics)

Ask how many grocery items to enter, collect names in a list, then display the numbered list and its length. Allow the user to replace one item by its displayed position; validate the position.

24. Score analyzer (list operations)

Collect a chosen number of quiz scores into a list. Report the scores, count, minimum, maximum, total, and average. Do not use `min`/`max` until the basic version has found them with a loop.

Extension: Sort a copy of the list and display the median.

25. Unique guest list

Collect names until the user types `done`. Keep a list without duplicate names, preserve insertion order, and allow a user to remove a name. Explain what happens if the name is not found.

26. Student profile (dictionary basics)

Create a dictionary for one student with ID, name, city, and course. Print each key/value pair, update the city, add an email field, and safely look up a key that may not exist.

27. Word frequency (dictionary counting)

Read a sentence, normalize it to lowercase, split it into words, and count each word in a dictionary. Print each word and count. Punctuation handling is an optional extension.

28. Inventory lookup (dictionary menu)

Use a dictionary whose keys are item codes and values are item names and prices. Repeatedly let a user look up an item, add an item, update a price, or quit. Handle missing codes and invalid choices.

29. Student marks register (list of dictionaries)

Store at least four students as dictionaries containing ID, name, and marks, inside a list. Print a roster, find a student by ID, and calculate the class average. Handle an unknown ID.

Extension: Find the highest scoring student and display their name and score.

30. Cafe order (nested data and totals)

Represent a menu as a dictionary mapping numeric IDs to item details. Collect selected items into an order list; each order entry should contain item name, unit price, and quantity. Calculate line totals and the subtotal, then print an itemized receipt. Reject unknown IDs and non-positive quantities.

Additional Focused Drills P1-P10

P1. Formatted receipt lines (output and variables)

Ask for a product name, unit price, and quantity. Print a tidy three-column receipt line using an f-string, alignment, and a calculated line total. Keep each input and calculated value in a clearly named variable.

P2. Four-operation practice (types and arithmetic)

Read two decimal numbers and print their sum, difference, product, quotient, and remainder where valid. Convert inputs to `float`, explain in comments why `%` needs a non-zero second value, and handle zero safely.

P3. Palindrome phrase (strings and slicing)

Read a word or phrase, normalize it to lowercase, remove spaces, and determine whether it reads the same forward and backward. Print the normalized text and result.

P4. Shipping cost selector (conditions and logic)

Read order amount and delivery distance. Apply a free-shipping rule when the order is at least Rs. 1000, otherwise charge Rs. 50 for up to 5 km and Rs. 100 beyond 5 km. Reject negative values and show the amount due.

P5. Multiples and count (for loop and condition)

Read a positive limit and a divisor. Loop from 1 through the limit, print all multiples of the divisor, and report their count and sum. Reject a divisor of zero.

P6. Guessing menu (while loop and input validation)

Choose a fixed secret number in the program. Keep asking the player to guess until they are correct or enter 0 to quit. Give higher/lower hints, count attempts, and reject guesses outside 1-100.

P7. Coordinate grid (nested loops and patterns)

Read a positive row and column count. Print a rectangular grid where each cell displays its row and column coordinates. Keep columns aligned and try a second version using row/column multiplication values.

P8. List statistics without helpers (list traversal)

Collect integers until the user types `done`. Use loops to calculate count, total, average, minimum, and maximum without using `sum`, `min`, or `max`. Handle an empty list.

P9. Contact directory (dictionary operations)

Use a dictionary to store names and phone numbers. Provide a repeating menu to add/update, search, delete, list all contacts, or quit. Handle missing names and empty directory safely.

P10. Class results (nested list/dictionary data)

Store at least three students in a list of dictionaries; each record includes a name and a list of three marks. Print each student's average and pass/fail result, then calculate the class average. Validate marks from 0 to 100.

Part 3: Integrated Capstone

31. Mini Stationery Shop

Build a console program inspired by `fund test/stationery_shop.py`. The finished program should combine as many skills from this pack as possible while remaining understandable to a beginner. Use only concepts introduced in `module-1` and `module-2`.

Starter data

Create a list of dictionaries with at least six products. Each product must have an ID, name, price, and stock quantity. Include at least one unavailable/out-of-stock product. Create a starting cash budget and a customer name prompt.

Required features

1. Show a formatted product catalogue using a `for` loop. Mark each product as available or out of stock based on its quantity.
2. Keep accepting product IDs and quantities until the customer chooses the documented finish option. Validate IDs, quantities, and stock; invalid requests must not change the order.
3. Store the order as a list of dictionaries. Each order line must preserve item name, unit price, quantity, and line total. If a product is selected more than once, either combine quantities into one line or clearly show separate lines.
4. After every valid item, show the updated subtotal. If the order is empty, offer a clear retry or exit path.
5. Ask for a payment method. Apply a 5% discount for online payment and no discount for cash. Treat any other response as invalid and reprompt or safely apply no discount with a clear message.
6. Calculate a 5% tax after discount, then the final bill. Use arithmetic and meaningful variable names.
7. Compare the final bill against the shop's starting budget. If the bill exceeds the budget, show a clear message and do not claim payment succeeded. Otherwise, show an itemized receipt with customer name, quantities, line totals, subtotal, discount, tax, final bill, and thank-you message.
8. Print an end-of-order summary: number of distinct order lines, total quantity, most expensive catalogue item, and total bill.
9. Keep input and output readable, use consistent currency formatting, and comment any non-obvious logic.

Optional challenges

- Add a membership discount but do not allow discounts to exceed a stated cap.

- Add a maximum order quantity and reject larger entries.

- Ask whether to start a new customer's order after printing the summary.

- Track remaining stock after a successful sale. Decide explicitly whether stock should be reduced when an order is rejected for exceeding budget.

- Add a daily sales total across multiple customers.

Completion checklist

[ ] Valid purchases create the correct order and totals.

[ ] Invalid product IDs and quantities are handled safely.

[ ] Out-of-stock items cannot be purchased.

[ ] Cash and online payment produce the expected discount.

[ ] Tax is computed after the discount.

[ ] Over-budget bills do not claim successful payment.

[ ] Empty orders do not crash or print a false receipt.

[ ] Summary values match the order.